Torneo de Komikan

Decisiones de diseño: sincronización y manejo de recursos

Sistemas Operativos • Juan Pablo Manríquez y Cristóbal Deck

1. Diseño del esquema de sincronización

1.1. Organización. El programa está escrito en C++17 y usa hilos POSIX dentro de un proceso, con `pthread_mutex_t` y `pthread_cond_t`. Hay un hilo por jugador (N), $C = \min(M, \lfloor N/2 \rfloor)$ hilos de mesa que actúan como juez, un receptor de señales y hasta 13 hilos de ficha por partida activa (Puma y doce Perros). El máximo nominal es $N + 14C + 2$ hilos, incluido el principal. Los recursos compartidos se protegen así:

Recurso	Primitiva	Por qué así
Parejas y estado del torneo	Mutex y condición del torneo	Registra las llegadas y permite esperar mesas sin consulta continua.
Límite de M partidas	Solo C hilos de mesa	Se cumple por construcción y las mesas se reutilizan.
Tablero de una partida	Mutex y condición por tablero	Una captura o movimiento es una operación indivisible.
Consola	Mutex de salida global	Evita que se mezclen los tableros de distintas mesas.

El primer jugador que registra su llegada queda pendiente y el segundo completa la pareja: el primero juega con el Puma y el segundo con los Perros, independientemente del ID. Cada mesa toma la pareja lista más antigua y, si no hay ninguna, espera con `pthread_cond_wait`. Si N es impar, el último que llegó queda sin pareja. Nunca hay más de M partidas simultáneas.

1.2. Movimientos y exclusión mutua. Cada ficha espera un retardo aleatorio (50–300 ms por defecto) mediante `pthread_cond_timedwait`, que puede despertarse al cerrar la partida. Las funciones `moverFicha` y `capturarPerro` comprueban bajo el mismo mutex origen, identidad, adyacencia, ocupación y resultado. Si dos fichas pretenden ocupar una casilla, solo una puede completar el movimiento; la otra vuelve a validar el tablero. La captura modifica origen, perro capturado y destino de forma atómica. Una victoria se marca como `fin` durante la operación que la produce. Cada perro tiene un identificador fijo, por lo que un hilo no puede mover otra ficha que ocupe la antigua casilla de un perro capturado.

1.3. Vida de una mesa y abandono. El trabajador de mesa es también el juez: inicializa el tablero, crea las fichas, comprueba la partida aproximadamente cada 100 ms y muestra estados periódicos. Tras victoria, abandono, límite temporal, SIGINT o fallo, despierta y une sus hilos con `pthread_join` antes de destruir los recursos. El abandono se notifica cooperativamente; `KOMIKAN_ABANDONA` y `KOMIKAN_FALLO_JUGADOR` permiten simularlo. `aceptarAbandono` y la victoria compiten bajo el mutex del tablero: prevalece la primera transición terminal aceptada. Si se acepta el abandono, gana el oponente y las otras mesas continúan.

1.4. Configuración y reproducibilidad. El archivo opcional permite configurar retardos, semilla, tiempo de partida y ubicación inicial de las trece fichas. Se valida antes de crear hilos: nodos 0–27, doce perros, sin posiciones superpuestas y valores numéricos válidos. Cada ficha utiliza una semilla propia derivada de la configuración; la planificación de hilos no es determinista aunque la semilla sea fija.

1.5. Salida y Ctrl+C. SIGINT se bloquea en los hilos y un receptor independiente la atiende con `sigwait`. Registra el cierre global, despierta jugadores y mesas, y permite terminar las fichas antes de los `join` y de destruir mutex y condiciones. En una interrupción normal se retorna 130. Los hilos comparten el espacio de direcciones, pero no se crean segmentos System V ni semáforos con nombre.

2. Análisis de riesgos de concurrencia

2.1. Condiciones de carrera. Casillas, capturas, identidades, `fin` y abandono se protegen con el mutex propio de cada tablero; las llegadas y asignaciones usan el del torneo. Las adyacencias se inicializan antes de crear fichas. Los saltos se generan con `todosLosSaltos()` cuando se necesitan, incluso dentro de las comprobaciones protegidas: no quedan precalculados en el tablero.

2.2. Deadlock. Las rutas que necesitan varios mutex evitan inversiones de adquisición y respetan una jerarquía compatible con torneo → salida → tablero. Se libera el mutex del tablero antes de adquirir el de salida cuando corresponde. Las esperas `pthread_cond_wait` y `pthread_cond_timedwait` liberan temporalmente su mutex asociado, y no se hace `join` reteniendo el mutex del torneo. Esto reduce el riesgo de ciclos, sin demostrar ausencia absoluta de bloqueos.

2.3. Livelock. Cuando un intento de movimiento falla, la ficha vuelve a dormir antes de reintentar; no hay un ciclo de consultas continuas sin espera. Los retardos aleatorios disminuyen reintentos sincronizados. El límite `maxSegundos` acota aproximadamente la duración de una partida que no logra terminar por victoria.

2.4. Starvation. Los mutex POSIX no garantizan un orden FIFO para las fichas. Las secciones críticas cortas y las pausas entre intentos reducen contención, pero no eliminan formalmente la posibilidad de inanición. En el servidor, las mesas atienden las parejas completas según su orden de llegada.

3. Evaluación y justificación del diseño

3.1. Alternativas consideradas.

Alternativa	Complejidad	Concurrencia	Decisión
Un mutex por casilla	Alta: capturar exige bloquear varias posiciones	Mayor dentro de la mesa	Descartada
Hilo árbitro separado	Alta: comunicación adicional y respuestas	Similar	Descartada
Hilo por partida y semáforo de M	Media: más hilos y control de cupos	Similar	Descartada
Mutex por tablero y C mesas	Baja: operaciones atómicas sencillas	Mesas independientes	**Elegida**

Se eligió un mutex por tablero porque simplifica las capturas y evita bloqueos parciales, a costa de serializar los movimientos dentro de cada partida. Tener solo C trabajadores limita por construcción las partidas simultáneas, sin crear una mesa por cada pareja.

3.2. Rendimiento y concurrencia (medido). La duración depende de los retardos y de la planificación; el mutex serializa movimientos de una mesa, pero las mesas funcionan de manera independiente. Con `minMs=20`, `maxMs=40` y `maxSegundos=1`, se observaron estos resultados en torneos cortos:

Escenario	Partidas	Tiempo total	Hilos máximos	RAM residente
$N = 20$, $M = 2$	10/10	3,24 s	50	4,77 MiB
$N = 40$, $M = 4$	20/20	3,27 s	98	5,15 MiB

Estas mediciones verifican el límite y la reutilización de mesas; no permiten afirmar aceleración lineal, porque cambiaron tanto N como M. Los escenarios de 100 y 1000 jugadores se analizaron solamente de forma estática.

3.3. Verificación y límites. Las 13 pruebas C++ del repositorio pasaron en la revisión final; el programa compila con `-Wall -Wextra -Werror -std=c++17 -pthread`. También se probaron SIGINT, abandonos, victorias y fallos controlados. No hay evidencia completa de ausencia de fugas o carreras con LeakSanitizer, ThreadSanitizer y Valgrind. Un `stdout` redirigido a un pipe lleno sin lector puede impedir finalizar rápidamente; escribir en un archivo regular evita ese bloqueo observado. Los diez saltos especiales del trono/techo corresponden a la interpretación geométrica del diagrama de la tarea.